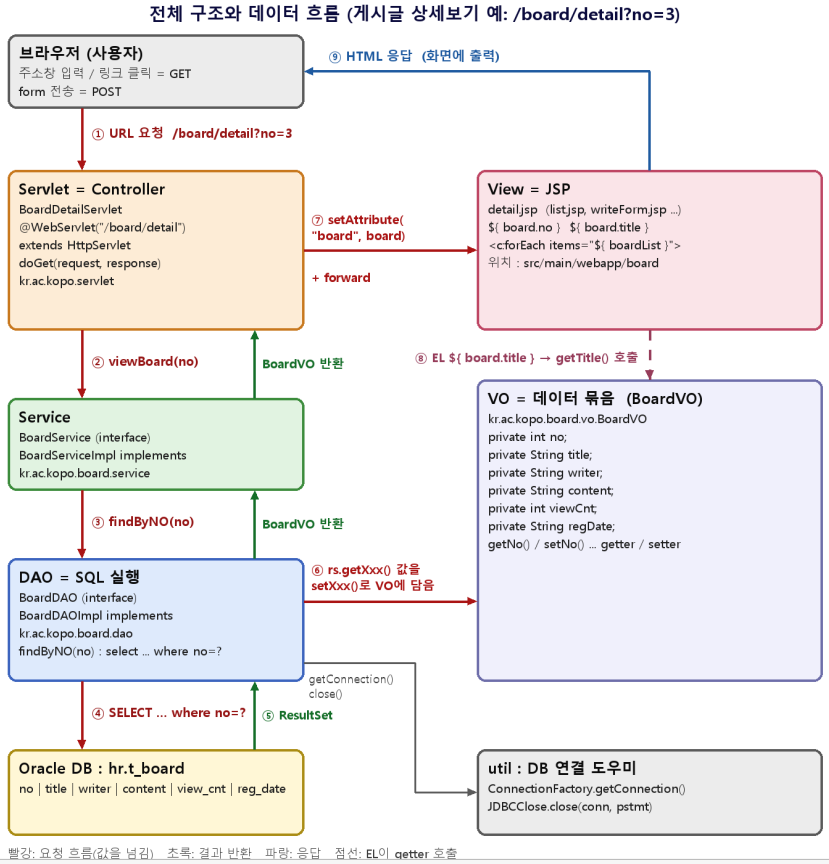
웹자바 게시판 (Model2 / MVC) 핵심 요약 - 수업자료 기반, 실행 검증 완료



패키지	kr.ac.kopo.util kr.ac.kopo.board.vo kr.ac.kopo.board.dao kr.ac.kopo.board.service kr.ac.kopo.servlet kr.ac.kopo.jdbc
파일 위치	.java 파일 : src/main/java/kr/ac/kopo/... jsp, html 파일 : src/main/webapp/board jar 3개 : src/main/webapp/WEB-INF/lib
상속/구현	public class BoardListServlet extends HttpServlet // @WebServlet("/board/list") public class BoardDAOImpl implements BoardDAO public class BoardServiceImpl implements BoardService boardDao = new BoardDAOImpl(); // Service가 DAO를 가짐

DAO 틀	Connection conn = null; PreparedStatement pstmt = null;
	try {
	conn = new ConnectionFactory().getConnection();
	pstmt = conn.prepareStatement(sql.toString());
	pstmt.setInt(1, no); // ? 번호는 1부터
	ResultSet rs = pstmt.executeQuery(); // select : while (rs.next())
	pstmt.executeUpdate(); // insert, update, delete
	} catch (Exception e) { e.printStackTrace(); }
	finally { JDBCClose.close(conn, pstmt); }

서블릿	int no = Integer.parseInt(request.getParameter("no"));
	request.setAttribute("board", board);
	request.getRequestDispatcher("/board/detail.jsp").forward(request, response);
	response.sendRedirect(request.getContextPath() + "/board/list");
	request.setCharacterEncoding("UTF-8"); // doPost 맨 위

JSP	<%@ taglib prefix="c" uri="jakarta.tags.core" %> ${ board.title } // getTitle() 호출 <c:forEach items="${ boardList }" var="board"> ... </c:forEach>
접속 정보	jdbc:oracle:thin:@localhost:1521/free user : hr password : hr // 시험 DB에 맞게 수정

2. 프로젝트 구조와 파일 위치 (Eclipse Project Explorer 화면)

Project Explorer	역할 / 설명
Exam-WEB	프로젝트 (Dynamic Web Project)
Deployment Descriptor: Exam-WEB	
JAX-WS Web Services	
Java Resources	자바 소스와 라이브러리 (실제 폴더는 src/main/java)
src/main/java	.java 파일이 들어가는 곳
kr.ac.kopo.board.dao	DAO : DB와 SQL 실행
BoardDAO.java	interface : save, findAll, findByNO, update, delete, increaseViewCnt
BoardDAOImpl.java	implements BoardDAO : JDBC + SQL
kr.ac.kopo.board.service	Service : DAO를 불러서 업무 처리
BoardService.java	interface : addBoard, getBoardList, getBoardByNo ...
BoardServiceImpl.java	implements BoardService : boardDao를 가짐
kr.ac.kopo.board.vo	VO : 게시글 한 건의 데이터 묶음
BoardVO.java	no, title, writer, content, viewCnt, regDate
kr.ac.kopo.jdbc	DB 접속 확인용
DBConnectionMain.java	main() : 접속 테스트
kr.ac.kopo.servlet	Controller : @WebServlet 클래스
BoardListServlet.java	/board/list : 목록
BoardDetailServlet.java	/board/detail : 상세
BoardWriteServlet.java	/board/write : 등록 (GET 폼, POST 저장)
BoardUpdateServlet.java	/board/update : 수정 (GET 폼, POST 저장)
BoardDeleteServlet.java	/board/delete : 삭제
kr.ac.kopo.util	공통 도우미
ConnectionFactory.java	getConnection() : 드라이버 로딩 + 접속
JDBCClose.java	close(conn, stmt) : 자원 해제
Libraries	
JRE System Library [JavaSE-21]	
Server Runtime [Apache Tomcat v11.0]	
Web App Libraries	WEB-INF/lib의 jar가 여기 표시됨
build	컴파일 결과 (건드리지 않음)
src	
main	
java	(Java Resources의 src/main/java와 같은 폴더)
webapp	html, jsp, css, js 가 들어가는 곳
board	게시판 화면 JSP
list.jsp	목록 화면 ← BoardListServlet이 forward
detail.jsp	상세 화면 ← BoardDetailServlet이 forward
writeForm.jsp	등록 폼 ← BoardWriteServlet(GET)
updateForm.jsp	수정 폼 ← BoardUpdateServlet(GET)
META-INF	
WEB-INF	보안 폴더 (URL로 직접 접근 불가)
lib	jar 3개를 붙여넣는 곳
ojdbc11.jar	Oracle JDBC 드라이버
jakarta.servlet.jsp.jstl-3.0.1.jar	JSTL 구현체
jakarta.servlet.jsp.jstl-api-3.0.2.jar	JSTL API
web.xml	환경설정 (@WebServlet을 쓰면 서블릿 등록 불필요)
index.html	메인 페이지 (링크로 /board/list 호출)

실제 Eclipse에서는 **src/main/java**가 Java Resources 아래에만 보이는 경우가 많습니다. 위 그림은 이해를 위해 두 위치를 모두 표시했습니다. 파일을 만드는 위치: **자바 클래스는 패키지 우클릭 → New → Class**, **JSP는 webapp/board 우클릭 → New → JSP File**.

3. DML (select, insert, update, delete)

수업자료 근거 강의 PDF 04 p12~13(전체조회, 새글등록), Mission-WEB.BoardDAOImpl(상세조회). 수정/삭제/조회수는 수업자료에 없어 같은 패턴으로 작성.

작업	SQL	JDBC	?에 넣는 값
전체조회	select no, title, writer, to_char(reg_date, 'yyyy-mm-dd') as reg_date from t_board order by no desc	executeQuery()	없음
상세조회	select no, title, writer, content, view_cnt, to_char(reg_date, 'yyyy-mm-dd') as reg_date from t_board where no = ?	executeQuery()	setInt(1, no)
등록 insert	insert into t_board(no, title, writer, content) values(seq_t_board_no.nextval, ?, ?, ?)	executeUpdate()	setString(1, title) setString(2, writer) setString(3, content)
수정 update	update t_board set title = ?, content = ? where no = ?	executeUpdate()	setString(1, title) setString(2, content) setInt(3, no)
삭제 delete	delete from t_board where no = ?	executeUpdate()	setInt(1, no)
조회수 update	update t_board set view_cnt = view_cnt + 1 where no = ?	executeUpdate()	setInt(1, no)

검증 이 PC의 DB(hr 계정)에서 확인한 것: t_board 컬럼은 no, title, writer, content, view_cnt, reg_date 이고, **title, writer, content는 NOT NULL**, view_cnt 기본값 0, reg_date 기본값 sysdate, 번호용 시퀀스 **seq_t_board_no**가 있습니다. 그래서 insert에서 no, view_cnt, reg_date는 넣지 않습니다.

주의 강의 PDF p13의 insert는 **values(?, ?, ?, ?)** 이고 **pstmt.setInt(1, board.getNo())**로 번호를 직접 넣습니다. 시퀀스가 없는 DB라면 이 방식을 쓰고, 이때는 폼에서 번호를 입력받아 board.setNo(...)를 호출해야 합니다. 시험 테이블 구조를 확인하세요.

주의 insert/update/delete 후 commit 코드는 필요 없습니다. JDBC는 기본 auto-commit이며, 삭제한 글이 다른 접속에서도 사라지는 것을 확인했습니다.

4. util 패키지 (kr.ac.kopo.util)

4-1. ConnectionFactory.java (DB 접속 정보가 있는 곳)

수업자료 근거 강의 PDF 04 p10, Mission-WEB, E: 폴더 ConnectionFactory (1).java. 수업 PDF는 **@localhost:1521:xe**, Mission-WEB은 **@localhost:1521/free**를 사용. 아래는 이 PC에서 실행 검증된 값(/free)입니다.

```
package kr.ac.kopo.util;

import java.sql.Connection;
import java.sql.DriverManager;

public class ConnectionFactory {

    public Connection getConnection() {

        Connection conn = null;
        try {
            // 1단계 : 오라클 드라이버 로딩
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // 2단계 : DB접속 후 Connection 객체 얻기
            String url      = "jdbc:oracle:thin:@localhost:1521/free";
            String user     = "hr";
            String password = "hr";

            conn = DriverManager.getConnection(url, user, password);

        } catch (Exception e) {
            e.printStackTrace();
        }

        return conn;
    }
}
```

4-2. JDBCClose.java

```
package kr.ac.kopo.util;

import java.sql.Connection;
import java.sql.Statement;

public class JDBCClose {

    public static void close(Connection conn, Statement stmt) {
        // 5단계 : DB연결 종료
        if (stmt != null) {
            try {
                stmt.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }

        if (conn != null) {
            try {
                conn.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```

4-3. DBConnectionMain.java (kr.ac.kopo.jdbc, 접속 확인용 main)

검증 Run As → Java Application 으로 실행하면 **conn : oracle.jdbc.driver.T4CConnection@...**가 출력됩니다. 이 PC에서 확인했습니다. **conn : null**이면 접속 실패이며 Console에 ORA- 오류가 같이 출력됩니다.

```
package kr.ac.kopo.jdbc;

import java.sql.Connection;

import kr.ac.kopo.util.ConnectionFactory;
import kr.ac.kopo.util.JDBCClose;

public class DBConnectionMain {
    public static void main(String[] args) {
        Connection conn = new ConnectionFactory().getConnection();
        System.out.println("conn : " + conn);

        JDBCClose.close(conn, null);
    }
}
```

5. VO (kr.ac.kopo.board.vo.BoardVO)

수업자료 근거 Mission-WEB의 BoardVO(필드 6개, 생성자 3개). EL 규칙(강의 PDF 03 p12): public 클래스, 매개변수 없는 생성자, private 멤버변수, getter/setter. ${ board.no }는 getNo()를 호출합니다.

```
package kr.ac.kopo.board.vo;

public class BoardVO {
    private int no;             // 번호
    private String title;       // 제목
    private String writer;      // 작성자
    private String content;     // 내용
    private int viewCnt;        // 조회수
    private String regDate;     // 작성일

    public BoardVO() {
        super();
    }

    public BoardVO(int no, String title, String writer, String regDate) {
        super();
        this.no = no;
        this.title = title;
        this.writer = writer;
        this.regDate = regDate;
    }

    public BoardVO(int no, String title, String writer, String content, int viewCnt, String regDate) {
        super();
        this.no = no;
        this.title = title;
        this.writer = writer;
        this.content = content;
        this.viewCnt = viewCnt;
        this.regDate = regDate;
    }

    public int getNo() {
        return no;
    }

    public void setNo(int no) {
        this.no = no;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getWriter() {
        return writer;
    }

    public void setWriter(String writer) {
        this.writer = writer;
    }

    public String getContent() {
```

```
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }

    public int getViewCnt() {
        return viewCnt;
    }

    public void setViewCnt(int viewCnt) {
        this.viewCnt = viewCnt;
    }

    public String getRegDate() {
        return regDate;
    }

    public void setRegDate(String regDate) {
        this.regDate = regDate;
    }

    @Override
    public String toString() {
        return "BoardVO [no=" + no + ", title=" + title + ", writer=" + writer + ", content=" + content
                + ", viewCnt=" + viewCnt + ", regDate=" + regDate + "]";
    }
}
```

VO 작성 요령: 테이블 컬럼마다 필드 하나. 컬럼 **view_cnt** → 필드 **viewCnt**(카멜 표기), 날짜는 to_char로 문자열로 받아 **String regDate**. getter/setter와 toString은 Eclipse **Source → Generate Getters and Setters... / Generate toString()**로 자동 생성.

6. DAO 인터페이스 (BoardDAO)

수업자료 근거 Mission-WEB의 BoardDAO는 save, findAll, findByNO. 강의 PDF는 같은 기능을 insertBoard / selectAllBoard 라는 이름으로 설명. update, delete, increaseViewCnt는 내가 추가한 메서드입니다.

```
package kr.ac.kopo.board.dao;

import java.util.List;

import kr.ac.kopo.board.vo.BoardVO;

public interface BoardDAO {

    /* 게시글 저장 */
    void save(BoardVO board);

    /* 전체게시글 조회 */
    List<BoardVO> findAll();

    /* 게시판 번호를 이용한 상세게시글 조회 */
    BoardVO findByNO(int no);

    /* 게시글 수정 */
    void update(BoardVO board);

    /* 게시글 삭제 */
    void delete(int no);

    /* 조회수 증가 */
    void increaseViewCnt(int no);
}
```

기능	메서드	반환	누가 부르나	SQL
목록	findAll()	List<BoardVO>	Service.getBoardList()	select (전체)
상세	findByNO(int no)	BoardVO (없으면 null)	Service.getBoardByNo(), viewBoard()	select where no=?
등록	save(BoardVO)	없음(void)	Service.addBoard()	insert
수정	update(BoardVO)	없음	Service.modifyBoard()	update
삭제	delete(int no)	없음	Service.removeBoard()	delete
조회수	increaseViewCnt(int no)	없음	Service.viewBoard()	update view_cnt

7. DAO 구현 (BoardDAOImpl) - 메서드별

7-0. 클래스 뼈대와 import

```
package kr.ac.kopo.board.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;
import java.util.List;

import kr.ac.kopo.board.vo.BoardVO;
import kr.ac.kopo.util.ConnectionFactory;
import kr.ac.kopo.util.JDBCClose;

public class BoardDAOImpl implements BoardDAO {

    // 아래 메서드들을 이 안에 넣는다

}
```

7-1. findAll() - 전체 조회 (select, 여러 행 : while)

강의 PDF p12, Mission-WEB.findAll. 행마다 BoardVO를 만들어 List에 add.

```
    @Override
    public List<BoardVO> findAll() {
        List<BoardVO> boardList = new ArrayList<>();

        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            conn = new ConnectionFactory().getConnection();

            StringBuilder sql = new StringBuilder();
            sql.append("select no, title, writer, to_char(reg_date, 'yyyy-mm-dd') as reg_date ");
            sql.append(" from t_board ");
            sql.append(" order by no desc ");

            pstmt = conn.prepareStatement(sql.toString());

            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                // 컬럼 추출
                int no          = rs.getInt("no");
                String title    = rs.getString("title");
                String writer   = rs.getString("writer");
                String regDate = rs.getString("reg_date");

                BoardVO board = new BoardVO();
                board.setNo(no);
                board.setTitle(title);
                board.setWriter(writer);
                board.setRegDate(regDate);

                // 리스트 추가
                boardList.add(board);
            }

        } catch (Exception e) {
            e.printStackTrace();
```

```
        } finally {
            JDBCClose.close(conn, pstmt);
        }

        return boardList;
    }
```

7-2. findByNO(int boardNo) - 상세 조회 (select, 한 행 : if)

Mission-WEB.findByNO. 결과가 있으면 BoardVO를 return, 없으면 마지막 return null.

```
    @Override
    public BoardVO findByNO(int boardNo) {

        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            conn = new ConnectionFactory().getConnection();

            // 3단계 : sql문 작성
            StringBuilder sql = new StringBuilder();
            sql.append("select no, title, writer, content, view_cnt, to_char(reg_date, 'yyyy-mm-dd') as reg_date ");
            sql.append(" from t_board ");
            sql.append(" where no = ? ");

            pstmt = conn.prepareStatement(sql.toString());
            pstmt.setInt(1, boardNo);

            // 4단계 : sql 결과 받아오기
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                BoardVO board = new BoardVO();
                board.setNo(rs.getInt("no"));
                board.setTitle(rs.getString("title"));
                board.setWriter(rs.getString("writer"));
                board.setContent(rs.getString("content"));
                board.setViewCnt(rs.getInt("view_cnt"));
                board.setRegDate(rs.getString("reg_date"));

                return board;
            }

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            JDBCClose.close(conn, pstmt);
        }

        return null;
    }
```

7-3. save(BoardVO) - 등록 (insert)

강의 PDF p13 insertBoard(). 시퀀스 사용은 3장 설명 참고.

```
    @Override
    public void save(BoardVO board) {

        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            conn = new ConnectionFactory().getConnection();
```

```
            StringBuilder sql = new StringBuilder();
            sql.append("insert into t_board(no, title, writer, content) ");
            sql.append(" values(seq_t_board_no.nextval, ?, ?, ?) ");

            pstmt = conn.prepareStatement(sql.toString());
            pstmt.setString(1, board.getTitle());
            pstmt.setString(2, board.getWriter());
            pstmt.setString(3, board.getContent());

            pstmt.executeUpdate();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            JDBCClose.close(conn, pstmt);
        }
    }
```

7-4. update(BoardVO) - 수정 (update)

수업자료에 없음. insert와 같은 executeUpdate 패턴.

```
    @Override
    public void update(BoardVO board) {

        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            conn = new ConnectionFactory().getConnection();

            StringBuilder sql = new StringBuilder();
            sql.append("update t_board ");
            sql.append(" set title = ?, content = ? ");
            sql.append(" where no = ? ");

            pstmt = conn.prepareStatement(sql.toString());
            pstmt.setString(1, board.getTitle());
            pstmt.setString(2, board.getContent());
            pstmt.setInt(3, board.getNo());

            pstmt.executeUpdate();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            JDBCClose.close(conn, pstmt);
        }
    }
```

7-5. delete(int no) - 삭제 (delete)

수업자료에 없음.

```
    @Override
    public void delete(int no) {

        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            conn = new ConnectionFactory().getConnection();

            StringBuilder sql = new StringBuilder();
            sql.append("delete from t_board ");
```

```
            sql.append(" where no = ? ");

            pstmt = conn.prepareStatement(sql.toString());
            pstmt.setInt(1, no);

            pstmt.executeUpdate();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            JDBCClose.close(conn, pstmt);
        }
    }
```

7-6. increaseViewCnt(int no) - 조회수 증가 (update)

수업자료에 없음. 상세보기 직전에 Service가 호출.

```
    @Override
    public void increaseViewCnt(int no) {

        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            conn = new ConnectionFactory().getConnection();

            StringBuilder sql = new StringBuilder();
            sql.append("update t_board ");
            sql.append(" set view_cnt = view_cnt + 1 ");
            sql.append(" where no = ? ");

            pstmt = conn.prepareStatement(sql.toString());
            pstmt.setInt(1, no);

            pstmt.executeUpdate();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            JDBCClose.close(conn, pstmt);
        }
    }
```

검증 select 두 개는 DB에서 읽은 값을 VO에 담아 반환하고, insert/update/delete 네 개는 실행 후 아무것도 반환하지 않습니다. 5단계(드라이버 로드, 접속, SQL 실행, 결과, 닫기)가 모든 메서드에서 같은 순서입니다.

8. Service (kr.ac.kopo.board.service)

수업자료 근거 Mission-WEB: BoardServiceImpl은 **private BoardDAO boardDao;** 필드를 두고 생성자에서 **new BoardDAOImpl()**로 만듭니다. 수업 코드에서는 addBoard()와 getBoardByNo()가 비어 있었습니다(TODO, null 반환). 아래는 구현을 채운 것입니다.

8-1. BoardService.java

```
package kr.ac.kopo.board.service;

import java.util.List;

import kr.ac.kopo.board.vo.BoardVO;

public interface BoardService {
    void addBoard(BoardVO board);          // 등록
    List<BoardVO> getBoardList();          // 전체조회
    BoardVO getBoardByNo(int no);          // 상세조회
    BoardVO viewBoard(int no);             // 조회수 증가 + 상세조회
    void modifyBoard(BoardVO board);       // 수정
    void removeBoard(int no);              // 삭제
}
```

8-2. BoardServiceImpl.java

```
package kr.ac.kopo.board.service;

import java.util.List;

import kr.ac.kopo.board.dao.BoardDAO;
import kr.ac.kopo.board.dao.BoardDAOImpl;
import kr.ac.kopo.board.vo.BoardVO;

public class BoardServiceImpl implements BoardService {

    private BoardDAO boardDao;

    public BoardServiceImpl() {
        boardDao = new BoardDAOImpl();
    }

    @Override
    public void addBoard(BoardVO board) {
        boardDao.save(board);
    }

    @Override
    public List<BoardVO> getBoardList() {
        return boardDao.findAll();
    }

    @Override
    public BoardVO getBoardByNo(int no) {
        return boardDao.findByNO(no);
    }

    @Override
    public BoardVO viewBoard(int no) {
        boardDao.increaseViewCnt(no);
        return boardDao.findByNO(no);
    }

    @Override
    public void modifyBoard(BoardVO board) {
        boardDao.update(board);
    }

    @Override
    public void removeBoard(int no) {
        boardDao.delete(no);
    }
}
```

viewBoard()는 조회수를 올린 뒤 상세를 조회합니다. 수정 화면 진입(**getBoardByNo**)에서는 조회수가 오르지 않아야 하므로 두 메서드를 나눴습니다.

9. 서블릿(Controller)과 @WebServlet 매핑

9-1. 매핑 규칙

수업자료 근거 수업 노트와 강의 PDF 01 p4~9: 서블릿은 HttpServlet을 상속, @WebServlet 어노테이션(Servlet 3.0+)으로 web.xml 없이 매핑, service()를 재정의하지 않으면 요청 방식에 따라 doGet / doPost가 호출됩니다. 수업 예: @WebServlet("/hi"), @WebServlet("/hiServlet").

```
@WebServlet("/board/detail")                    // <- 이 주소로 요청이 오면
public class BoardDetailServlet extends HttpServlet {   // <- 이 클래스가 실행된다 (HttpServlet 상속 필수)

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        ...
    }
}

브라우저 주소 = http://localhost:8080 / Exam-WEB / board/detail ? no=3
                [서버:포트]     [프로젝트명]   [@WebServlet 값]  [파라미터]
```

요청을 보내는 방법	HTTP 방식	서블릿에서 받는 메서드
주소창에 직접 입력, <a href="..."> 링크 클릭, response.sendRedirect()	GET	**doGet**(request, response)
<form action="..." method="post"> 의 전송 버튼	POST	**doPost**(request, response)
doGet/doPost 대신 service()를 재정의 (수업의 HiServlet)	둘 다	service(request, response) - GET/POST 구분 없이 모두 처리

항목	내용
여러 주소 연결	@WebServlet({"/hello", "/hi"}) (강의 PDF 01 p7)
같은 주소에 GET과 POST	하나의 서블릿에 doGet과 doPost를 둘 다 작성 (BoardWriteServlet: GET = 폼 보여주기, POST = 저장)
주소는 서로 달라야 함	다른 서블릿에 같은 값을 쓰면 서버 시작 시 오류. 각 서블릿마다 다른 주소 사용
JSP 안에서 주소 만들기	${ pageContext.request.contextPath }/board/list : 프로젝트명(Exam-WEB)을 직접 쓰지 않기 위함
404 오류 확인	(1) 주소 철자, 앞에 / (2) 프로젝트명 (3) 서블릿을 새로 만든 뒤 서버 재시작 (4) 패키지/어노테이션 import 확인
web.xml 방식 (대안)	<servlet><servlet-name>..., <servlet-class>전체경로..., <servlet-mapping><url-pattern>/board/list ... (강의 PDF 01 p6). @WebServlet을 쓰면 필요 없음

9-2. 주소와 서블릿 대응표 (이 프로젝트)

@WebServlet 주소	서블릿 클래스	doGet이 하는 일	doPost가 하는 일	이동
/board/list	BoardListServlet	목록 조회 → boardList 등록	-	forward list.jsp
/board/detail	BoardDetailServlet	?no= 로 상세 조회(+조회수) → board 등록	-	forward detail.jsp
/board/write	BoardWriteServlet	등록 폼 보여주기	파라미터 → VO → insert	GET: forward writeForm.jsp POST: redirect /board/list
/board/update	BoardUpdateServlet	기존 글 조회 → 폼에 채움	파라미터 → VO → update	GET: forward updateForm.jsp POST: redirect /board/detail?no=
/board/delete	BoardDeleteServlet	?no= 글 삭제	-	redirect /board/list

9-3. BoardListServlet.java - 목록

```
package kr.ac.kopo.servlet;

import java.io.IOException;
import java.util.List;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import kr.ac.kopo.board.service.BoardService;
import kr.ac.kopo.board.service.BoardServiceImpl;
import kr.ac.kopo.board.vo.BoardVO;

@WebServlet("/board/list")
public class BoardListServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // 1. DB에서 데이터 가져오기
        BoardService boardService = new BoardServiceImpl();
        List<BoardVO> list = boardService.getBoardList();

        // 2. 공유영역에 boardList라는 이름으로 등록
        request.setAttribute("boardList", list);

        // 3. 화면(JSP)으로 이동 (forward)
        request.getRequestDispatcher("/board/list.jsp").forward(request, response);
    }
}
```

주의 나머지 서블릿은 같은 import를 씁니다(jakarta.servlet.* 와 kr.ac.kopo.board.*). Eclipse에서 **Ctrl + Shift + O**(Organize Imports)를 누르면 자동으로 채워지며, jakarta와 javax 중에서는 **jakarta.servlet...**을 선택해야 합니다.

9-4. BoardDetailServlet.java - 상세

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import kr.ac.kopo.board.service.BoardService;
import kr.ac.kopo.board.service.BoardServiceImpl;
import kr.ac.kopo.board.vo.BoardVO;

@WebServlet("/board/detail")
public class BoardDetailServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        /*
        http://localhost:8080/Exam-WEB/board/detail?no=3 요청이 오면,

        1. 게시판번호 파라미터(no)추출
        2. 번호에 해당하는 게시글 조회
        3. 조회된 게시글을 공유영역에 등록
        */

        // String > int로 변환 : Integer.parseInt(data);
        String strNo = request.getParameter("no");
        int boardNo = Integer.parseInt(strNo);

        BoardService boardService = new BoardServiceImpl();
        BoardVO board = boardService.viewBoard(boardNo);

        request.setAttribute("board", board);

        request.getRequestDispatcher("/board/detail.jsp").forward(request, response);
    }
}
```

9-5. BoardWriteServlet.java - 등록 (GET 폼 / POST 저장)

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import kr.ac.kopo.board.service.BoardService;
import kr.ac.kopo.board.service.BoardServiceImpl;
import kr.ac.kopo.board.vo.BoardVO;

@WebServlet("/board/write")
public class BoardWriteServlet extends HttpServlet {

    // 글쓰기 화면 보여주기 (주소창, 링크 클릭 = GET)
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.getRequestDispatcher("/board/writeForm.jsp").forward(request, response);
    }

    // 등록 처리 (form method="post" = POST)
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");      // getParameter 보다 먼저!

        BoardVO board = new BoardVO();
        board.setTitle(request.getParameter("title"));
        board.setWriter(request.getParameter("writer"));
        board.setContent(request.getParameter("content"));

        BoardService boardService = new BoardServiceImpl();
        boardService.addBoard(board);

        // 등록 후 목록 서블릿으로 다시 요청 (redirect)
        response.sendRedirect(request.getContextPath() + "/board/list");
    }
}
```

9-6. BoardUpdateServlet.java - 수정 (GET 폼 / POST 저장)

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import kr.ac.kopo.board.service.BoardService;
import kr.ac.kopo.board.service.BoardServiceImpl;
import kr.ac.kopo.board.vo.BoardVO;

@WebServlet("/board/update")
public class BoardUpdateServlet extends HttpServlet {

    // 수정 화면 : 기존 내용을 조회해서 폼에 채워 보여주기
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        int no = Integer.parseInt(request.getParameter("no"));

        BoardService boardService = new BoardServiceImpl();
        BoardVO board = boardService.getBoardByNo(no);      // 조회수는 올리지 않음

        request.setAttribute("board", board);
        request.getRequestDispatcher("/board/updateForm.jsp").forward(request, response);
    }

    // 수정 처리
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");

        BoardVO board = new BoardVO();
        board.setNo(Integer.parseInt(request.getParameter("no")));
        board.setTitle(request.getParameter("title"));
        board.setContent(request.getParameter("content"));

        BoardService boardService = new BoardServiceImpl();
        boardService.modifyBoard(board);

        response.sendRedirect(request.getContextPath() + "/board/detail?no=" + board.getNo());
    }
}
```

9-7. BoardDeleteServlet.java - 삭제

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import kr.ac.kopo.board.service.BoardService;
import kr.ac.kopo.board.service.BoardServiceImpl;

@WebServlet("/board/delete")
public class BoardDeleteServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        int no = Integer.parseInt(request.getParameter("no"));

        BoardService boardService = new BoardServiceImpl();
        boardService.removeBoard(no);

        response.sendRedirect(request.getContextPath() + "/board/list");
    }
}
```

검증 위 5개 서블릿을 Tomcat 11.0.25에서 실제로 호출해 확인했습니다(10장 검증 결과). forward는 주소창이 바뀌지 않고, redirect는 주소창이 새 주소로 바뀝니다. 등록/수정/삭제 후 redirect를 쓰는 이유는 새로고침 시 같은 요청이 다시 실행되는 것을 막기 위해서입니다.

10. HTML, JSP, 서블릿, DB 연동 (동적 페이지 만들기)

10-1. 한눈에 보는 연동 방법

연결	쓰는 방법	프로젝트에서의 예
html → 서블릿	<a href="board/list"> (html은 정적 파일이라 서블릿 주소로 이동만 함)	index.html 의 "게시판 목록 보기" 링크
JSP 링크 → 서블릿	<a href="${ pageContext.request.contextPath }/board/detail?no=${ board.no }"> ?no= 뒤 값을 DB에서 읽은 값으로 만듦 = 동적	list.jsp 의 제목 링크
JSP form → 서블릿	<form action=".../board/write" method="post"> 안의 <input name="title"> ...	writeForm.jsp, updateForm.jsp
form 값 → 서블릿	request.getParameter("title") - 괄호 안은 input의 name 값과 정확히 같아야 함	BoardWriteServlet.doPost
서블릿 → JSP	request.setAttribute("board", board); request.getRequestDispatcher("/board/detail.jsp").forward(request, response);	BoardDetailServlet.doGet
JSP에서 값 꺼내기	${ board.title }, ${ boardList } (EL) / <c:forEach>, <c:out> (JSTL)	detail.jsp, list.jsp
서블릿 → 서블릿	response.sendRedirect(request.getContextPath() + "/board/list");	등록/수정/삭제 후

10-2. DB와 연동하는 동적 페이지가 만들어지는 순서

순서	하는 일	코드
1	브라우저가 서블릿 주소를 요청	GET /Exam-WEB/board/detail?no=3
2	서블릿이 파라미터를 꺼냄	String strNo = request.getParameter("no");
3	문자열을 숫자로 변환	int boardNo = Integer.parseInt(strNo);
4	Service, DAO가 DB에서 조회해 VO에 담음	BoardVO board = boardService.viewBoard(boardNo);
5	VO를 공유영역(request)에 올림	request.setAttribute("board", board);
6	JSP로 forward	request.getRequestDispatcher("/board/detail.jsp").forward(request, response);
7	JSP가 EL로 꺼내서 HTML을 만듦	${ board.title }
8	만들어진 HTML이 브라우저에 응답으로 전달	(화면 출력)

10-3. 수업 코드 detail.jsp 핵심 (Mission-WEB 원본)

수업자료 근거 Mission-WEB/src/main/webapp/board/detail.jsp 원본에서 핵심만 뽑은 것입니다. 수업에서는 이 JSP가 서블릿 역할(1~3단계)까지 직접 했습니다(Model1). Model2에서는 이 부분이 서블릿으로 이동하고, JSP에는 아래 5번(EL 출력)만 남습니다.

```
<%
    // (1) 파라미터 추출 : 요청 주소  detail.jsp?no=3  의 no 값
    String strNo = request.getParameter("no");

    // (2) String > int 변환
    int boardNo = Integer.parseInt(strNo);
```

```
    // (3) 번호에 해당하는 게시글 조회 (Service -> DAO -> DB)
    BoardService boardService = new BoardServiceImpl();
    BoardVO board = boardService.getBoardByNo(boardNo);

    // (4) 공유영역에 "board"라는 이름으로 등록
    pageContext.setAttribute("board", board);
%>

<!-- (5) EL로 꺼내서 출력 -->
<td>${ board.no }</td>
<td>${ board.title }</td>
<td>${ board.writer }</td>
<td>${ board.content }</td>
<td>${ board.viewCnt }</td>
<td>${ board.regDate }</td>
```

단계	수업 detail.jsp (Model1)	Model2 로 옮기면	설명
(1)	request.getParameter("no")	BoardDetailServlet.doGet 안	URL의 ?no=3 에서 "3"(문자열)을 꺼냄. 파라미터 이름은 링크의 ?no= 와 같아야 함
(2)	Integer.parseInt(strNo)	BoardDetailServlet.doGet 안	문자열 → int. 숫자가 아니면 NumberFormatException
(3)	boardService.getBoardByNo(boardNo)	BoardDetailServlet.doGet 안 (viewBoard)	DB 조회. 결과 BoardVO (없으면 null)
(4)	pageContext.setAttribute("board", board)	request.setAttribute("board", board)	이름 "board"가 JSP의 ${ board } 이름이 된다. forward 하려면 request 영역
(5)	${ board.title }	그대로 JSP에 남음	board 객체의 getTitle() 호출. 이름 규칙: 필드 title → getTitle()

주의 EL이 객체를 찾는 순서는 pageContext → request → session → application (강의 PDF 03 p7). 서블릿에서 request에 올리면 JSP의 ${ board }가 request 영역에서 찾습니다. 이름이 같은 객체가 여러 영역에 있으면 ${ requestScope.board }로 지정합니다.

주의 수업 원본 detail.jsp는 **getBoardByNo()**가 null을 반환하는 미완성 상태였습니다(BoardServiceImpl TODO). 그대로 두면 화면이 비어 나옵니다. 시험 코드에서는 8장처럼 Service/DAO를 완성해야 합니다.

10-4. 목록 화면(list.jsp)에서 쓰는 JSTL 반복

```
<c:forEach items="${ boardList }" var="board">      // boardList(List<BoardVO>)를 한 건씩 board에 담아 반복
    <tr>
        <td>${ board.no }</td>
        <td><a href="${ pageContext.request.contextPath }/board/detail?no=${ board.no }">
                <c:out value="${ board.title }" /></a></td>
        <td>${ board.writer }</td>
        <td>${ board.regDate }</td>
    </tr>
</c:forEach>
```

11. 화면 파일 전체 (검증된 코드)

11-1. index.html (src/main/webapp)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Main Page</title>
</head>
<body>
    <h1>Main Page</h1>

    <!-- html 에서 서블릿으로 이동 : 서블릿 주소(@WebServlet 값)를 href에 -->
    <a href="board/list">게시판 목록 보기</a>
</body>
</html>
```

11-2. board/list.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>게시판</title>
</head>
<body>
    <div align="center">
        <hr>
        <h2>전체 게시글</h2>
        <hr>

        <table border="1" width="80%">
            <tr>
                <th width="7%">번호</th>
                <th>제목</th>
                <th width="17%">작성자</th>
                <th width="23%">등록일</th>
            </tr>

            <%-- 서블릿이 request에 올려둔 boardList를 꺼내서 반복 --%>
            <c:forEach items="${ boardList }" var="board">
                <tr>
                    <td>${ board.no }</td>
                    <td>
                        <a href="${ pageContext.request.contextPath }/board/detail?no=${ board.no }">
                            <c:out value="${ board.title }" />
                        </a>
                    </td>
                    <td>${ board.writer }</td>
                    <td>${ board.regDate }</td>
                </tr>
            </c:forEach>
        </table>

        <br>
        <a href="${ pageContext.request.contextPath }/board/write">글쓰기</a>
    </div>
</body>
</html>
```

11-3. board/detail.jsp

```jsp
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>상세페이지</title>
</head>
<body>
    <div align="center">
        <hr>
        <h2>상세페이지</h2>
        <hr>

        <%-- 서블릿이 request에 올려둔 board(BoardVO)를 EL로 꺼내서 출력 --%>
        <table border="1" style="width: 80%">
            <tr>
                <th style="width: 25%">번호</th>
                <td>${ board.no }</td>
            </tr>
            <tr>
                <th>제목</th>
                <td><c:out value="${ board.title }" /></td>
            </tr>
            <tr>
                <th>작성자</th>
                <td>${ board.writer }</td>
            </tr>
            <tr>
                <th>내용</th>
                <td><c:out value="${ board.content }" /></td>
            </tr>
            <tr>
                <th>조회수</th>
                <td>${ board.viewCnt }</td>
            </tr>
            <tr>
                <th>작성일</th>
                <td>${ board.regDate }</td>
            </tr>
        </table>

        <br>
        <a href="${ pageContext.request.contextPath }/board/list">목록</a>
        <a href="${ pageContext.request.contextPath }/board/update?no=${ board.no }">수정</a>
        <a href="${ pageContext.request.contextPath }/board/delete?no=${ board.no }">삭제</a>
    </div>
</body>
</html>
```

11-4. board/writeForm.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>글쓰기</title>
</head>
<body>
    <div align="center">
        <hr>
        <h2>새 글 등록</h2>
        <hr>

        <%-- action : 서블릿 주소 / name : 서블릿의 request.getParameter("...") 이름과 같아야 함 --%>
        <form action="${ pageContext.request.contextPath }/board/write" method="post">
            제목   <input type="text" name="title"><br>
            작성자 <input type="text" name="writer"><br>
            내용   <textarea name="content" rows="5" cols="40"></textarea><br>
            <input type="submit" value="등록">
        </form>

        <br>
        <a href="${ pageContext.request.contextPath }/board/list">목록</a>
    </div>
</body>
</html>
```

11-5. board/updateForm.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>글 수정</title>
</head>
<body>
    <div align="center">
        <hr>
        <h2>글 수정</h2>
        <hr>

        <form action="${ pageContext.request.contextPath }/board/update" method="post">
            <input type="hidden" name="no" value="${ board.no }">
            제목 <input type="text" name="title" value="${ board.title }"><br>
            내용 <textarea name="content" rows="5" cols="40">${ board.content }</textarea><br>
            <input type="submit" value="수정">
        </form>

        <br>
        <a href="${ pageContext.request.contextPath }/board/detail?no=${ board.no }">취소</a>
    </div>
</body>
</html>
```

12. 실행 검증 결과

아래는 이 PC에서 실제로 실행한 결과입니다. 위 문서의 모든 Java/JSP 코드는 이 검증에 사용된 파일에서 그대로 가져왔습니다.

항목	내용
컴파일	JDK 21.0.12 javac 로 13개 클래스 오류 없음 (경고: 서블릿에 serialVersionUID 없음 - Eclipse에서 노란 경고로 보이지만 무시 가능)
서버	Apache Tomcat 11.0.25 (D:₩Lecture₩bin 의 것), 포트 8081에서 실행. 검증 후 종료
DB	Oracle Database 26ai Free, localhost:1521/free, 계정 hr/hr, 테이블 t_board
DBConnectionMain	conn : oracle.jdbc.driver.T4CConnection@... 출력 확인
총 14개 항목	**14개 모두 통과** (아래 표)

#	확인한 동작	결과
1	GET /index.html 200, board/list 링크 존재	통과
2	GET /board/list 200, 기존 글 2건 표시	통과
3	GET /board/write 등록 폼 표시 (method=post, name=title)	통과
4	POST /board/write → 302 redirect /board/list	통과
5	목록에 새 글 추가 (한글 제목 정상)	통과
6	GET /board/detail?no=N 제목/내용/작성자 표시	통과
7	첫 조회 시 조회수 1	통과
8	다시 조회 시 조회수 2	통과
9	상세 화면에 수정/삭제 링크	통과
10	GET /board/update?no=N 기존 값이 폼에 채워짐	통과
11	POST /board/update → 302 redirect /board/detail?no=N	통과
12	수정한 제목/내용이 상세에 반영	통과
13	GET /board/delete?no=N → 302 redirect /board/list	통과
14	삭제 후 목록이 처음 상태(2건)로 복구	통과

주의 검증 중 DB를 건드린 부분: 테스트용 글 1건을 등록, 수정, 삭제했고 기존 2건의 데이터(제목, 조회수)는 그대로입니다. 다만 등록 시 시퀀스 seq_t_board_no 값이 1 증가했습니다(다음 글 번호가 하나 건너뛰어짐).

검증하지 못한 것

항목	이유
Eclipse 안에서의 실행 (Run on Server)	Eclipse 화면은 직접 조작하지 못함. Tomcat을 직접 실행해 확인. 프로젝트 구조와 경로는 Mission-WEB과 같게 구성
시험 DB의 테이블 구조	시험지의 테이블이 t_board와 다르면 컬럼/SQL을 맞춰야 함
시험 DB의 서비스 이름	이 PC는 /free. 수업 PDF는 :xe. 시험 환경에 맞게 ConnectionFactory url 수정
페이징, 검색, 로그인, 파일첨부	수업자료에 없고 요구 여부를 모름

검증된 소스 전체: **D:₩Lecture₩exam_test₩Exam-WEB-검증완료-소스** (src 폴더).